

Fehlerbehandlung und UnitTests

Debugging

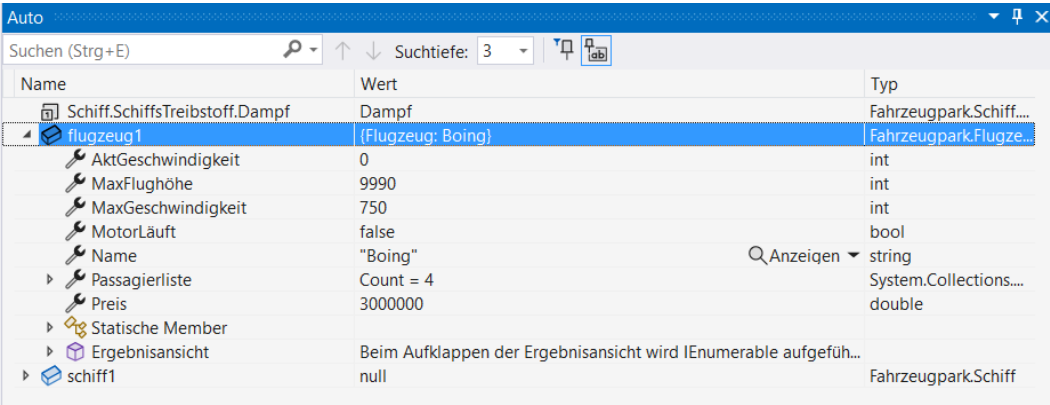
- Haltepunkte pausieren die Ausführung BEVOR die gewählte Zeile ausgeführt wird



```
114  
115 Flugzeug flugzeug1 = new Flugzeug("Boing", 750, 3000000, 9990);  
116 Schiff schiff1 = new Schiff("Titanic", 40, 3500000, Schiff.SchiffsTreibstoff.Dampf);  
117 BeladeFahrzeuge(pkwl, flugzeug1);
```

-  Einzelschritt (F11): Schrittweise Ausführung mit Sprung in Methodendefinitionen
-  Prozedurschritt (F10): Schrittweise Ausführung auf aktueller Ebene

- Auto- und Lokalfenster beinhalten aktuelle Variablenbelegungen



Name	Wert	Typ
Schiff.SchiffsTreibstoff.Dampf	Dampf	Fahrzeugpark.Schiff...
flugzeug1	{Flugzeug: Boing}	Fahrzeugpark.Flugze...
AktGeschwindigkeit	0	int
MaxFlughöhe	9990	int
MaxGeschwindigkeit	750	int
MotorLäuft	false	bool
Name	"Boing"	string
Passagierliste	Count = 4	System.Collections....
Preis	3000000	double
Statische Member		
Ergebnisansicht	Beim Aufklappen der Ergebnisansicht wird IEnumerable aufgeföh...	
schiff1	null	Fahrzeugpark.Schiff

Fehlerbehandlung

- mit einem try-catch-Block können Fehler abgefangen und behandelt werden
- try-Block
 - eigentliche Anwendung
- catch-Block
 - Fehler werden abgefangen und behandelt
- finally-Block
 - ist optional
 - wird garantiert ausgeführt

```
try
{
    int Eingabe = int.Parse(Console.ReadLine());
}
catch (FormatException e)
{
    Console.WriteLine("Bitte eine Zahl eingeben.");
    return;
}
catch (Exception e)
{
    Console.WriteLine(e.Message);
    return;
}
finally
{
    Console.WriteLine("Danke");
}
```

eigene Exception erstellen

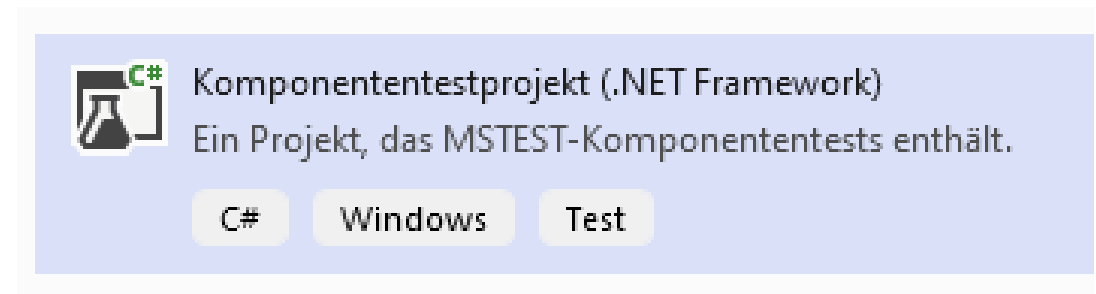
- jede neue Exception ist von der Klasse „Exception“ abgeleitet
- durch „catch (Exception)“ werden somit alle nicht zuvor abgefangenen Fehlermeldungen behandelt

```
public class MyException : Exception
{
    public MyException(string message) : base(message)
    {
    }
}
```

```
try
{
    //einen Fehler werfen
    throw new MyException("Eine MyException wurde geworfen!");
}
catch (MyException e) //über ein catch-Block den Fehler abfangen
{
    Console.WriteLine(e.Message);
    //Eine MyException wurde geworfen!
}
catch (Exception e) //alle anderen Exceptions werden abgefangen
{
    Console.WriteLine(e.Message);
}
finally
{
    Console.WriteLine("try-catch-Block ist fertig..");
}
```

UnitTest - Komponententest

- mit einem UnitTest können Funktionen auf eine definierte Eingabe und eine erwartete Rückgabe getestet werden
- ein extra Projekt, welches in VisualStudio erstellt werden kann



Testklasse stellen

- eine Klasse muss mit dem Attribut „[TestClass]“ versehen werden, damit VS diese als Testklasse erkennt
- eine Methode muss mit dem Attribut „[TestMethod]“ versehen werden, damit VS diese als Testmethode erkennt und bei einem UnitTest aufruft

```
[TestClass]
0 Verweise
public class UnitTest1
{
    [TestMethod]
    0 Verweise
    public void UnitTestKlasseHalbieren()
    {
        UnitTestKlasse unitTestKlasse = new UnitTestKlasse();
        for (int i = 0; i < 100; i++)
        {
            int halbiert = unitTestKlasse.Halbieren(i);
            Assert.AreEqual(i / 2, halbiert);
        }
    }
}
```

UnitTest ausführen

- um einen UnitTest zu starten klickt man im Menü „Test“ auf den Punkt „Alle Test ausführen“
- alternativ mithilfe der Tastenkombination [STRG]+[R] [STRG]+[A]
- im Test-Explorer werden dann die Ergebnisse angezeigt

